GAME PROGRAMMING COURSE — UNITY

Unityで避けたい書き方

AVOID

エラーは出ないのに、重くなる・動きがおかしくなる・壊れやすくなる 20 の書き方

対象：1年生

はじめに

動くけど、避けたい書き方

ここで紹介する 20 個は、どれもエラーにならず一見ちゃんと動く。だから気づきにくい。

重くなる

毎フレームの無駄な処理でカクつく。敵や弾が増えるほど目立ってくる。

動きがおかしくなる

PC の性能やタイミングによって、速さや挙動が変わってしまう。

壊れやすい

名前や順番を変えただけで、エラーも出ずに黙って動かなくなる。

PART 1 | NG 01 – 06

毎フレームの無駄

Update は1秒に何十回も呼ばれる。そこに重い処理を置かない

1

NG 01 ｜ 重くなる・壊れやすい

Update の中で毎回 Find する

C# EnemyChaser.cs NG

```
1 void Update()
2 {
3     GameObject player = GameObject.Find("Player");
4     transform.LookAt(player.transform);
5 }
```

C# EnemyChaser.cs OK

```
1 Transform target;
2
3 void Start()
4 {
5     target = GameObject.Find("Player").transform;
6 }
7
8 void Update()
9 {
10     transform.LookAt(target);
11 }
```

何が起きるか

Find はシーン中のオブジェクトを名前で探す処理。敵が 10 体いれば1秒に約 600 回探すことになる。

直し方

Start で1回だけ探して変数に入れる。Inspector でドラッグして渡せば Find 自体いらない。

NG 02 ｜ 重くなる

Update の中で毎回 GetComponent する

C# PlayerSpriteFlipper.cs NG

```
1 void Update()
2 {
3     float inputX = Input.GetAxis("Horizontal");
4     if (inputX == 0f) return;
5     GetComponent<SpriteRenderer>().flipX =
6         inputX < 0f;
7 }
```

C# PlayerSpriteFlipper.cs OK

```
1 SpriteRenderer spriteRenderer;
2
3 void Awake()
4 {
5     spriteRenderer = GetComponent<SpriteRenderer>();
6 }
7 void Update()
8 {
9     float inputX = Input.GetAxis("Horizontal");
10    if (inputX == 0f) return;
11    spriteRenderer.flipX = inputX < 0f;
12 }
```

何が起きるか

GetComponent は付いているコンポーネントの中から探す処理。毎フレーム同じものを探し直している。

直し方

Awake で1回だけ取って変数に入れておく。あとはその変数を使うだけ。

NG 03 ｜ 重くなる

UI の文字を毎フレーム書きかえる

C# ScoreDisplay.cs NG

```
1 [SerializeField] TMP_Text scoreText;
2 int score;
3
4 void Update()
5 {
6     scoreText.text = "Score: " + score;
7 }
```

何が起きるか

文字をつなぐたびに新しい文字列が作られ、ゴミがたまる。たまると掃除（GC）で一瞬止まる。

C# ScoreDisplay.cs OK

```
1 [SerializeField] TMP_Text scoreText;
2 int score;
3
4 public void AddScore(int amount)
5 {
6     score += amount;
7     scoreText.text = "Score: " + score;
8 }
```

直し方

表示はスコアが変わったときだけ更新する。スコアを変える処理を1つのメソッドにまとめる。

NG 04 ｜ 重くなる

Instantiate と Destroy を連発する

C# PlayerShooter.cs　NG

```
1 void Update()
2 {
3     if (Input.GetKey(KeyCode.Z))
4     {
5         GameObject bullet = Instantiate(bulletPrefab
6             muzzle.position, Quaternion.identity);
7         Destroy(bullet, 2f);
8     }
9 }
```

何が起きるか

押している間、1秒に約 60 発作って 60 発壊す。生成と削除は重い処理で、ゴミもたまる。

OK の考え方：使い回す（オブジェクトプール）

1. 最初に弾をまとめて作り、非表示にしておく
2. 撃つときは、非表示の弾を表示して使う
3. 当たったら Destroy せず、非表示に戻す
4. 次に撃つとき、その弾をもう一度使う

直し方

まず発射間隔をつけて数を減らす。大量に出すなら、作った弾を使い回す。

NG 05 ｜ 重くなる

Debug.Log を出しっぱなしにする

C# PlayerMovement.cs NG

```
1 void Update()
2 {
3     Debug.Log("x = " + transform.position.x);
4     Move();
5 }
```

C# PlayerMovement.cs OK

```
1 void Update()
2 {
3     Move();
4 }
5
6 void OnTriggerEnter2D(Collider2D other)
7 {
8     Debug.Log("ゴールに入った");
9 }
```

何が起きるか

ログの出力は重い。ビルドしたゲームでもログファイルに書かれ続け、大事なエラーも埋もれる。

直し方

確認が終わったら消す。残すなら「何かが起きたときだけ」出すようにする。

NG 06 | 重くなる

空の Start・Update を残す

C# GoalTrigger.cs NG

```
1 void Start()
2 {
3 }
4
5 void Update()
6 {
7 }
8
9 void OnTriggerEnter2D(Collider2D other)
10 {
11     ShowClear();
12 }
```

C# GoalTrigger.cs OK

```
1 void OnTriggerEnter2D(Collider2D other)
2 {
3     ShowClear();
4 }
```

何が起きるか

中身が空でも、Update があれば Unity は毎フレーム呼び出す。読む人も「何かあるのか」と探してしまう。

直し方

使っていない Start・Update は消す。スクリプトを作ったら、最初に不要な部分を整理する。

PART 2 | NG 07 – 11

時間・物理・入力

2

自分の PC では動くのに、ほかの PC では動きが変わってしまう

NG 07 ｜ 動きがおかしくなる

移動に Time.deltaTime を掛けない

C# PlayerMovement.cs NG

```
1 void Update()
2 {
3     float inputX = Input.GetAxis("Horizontal");
4     transform.Translate(inputX * 0.1f, 0f, 0f);
5 }
```

C# PlayerMovement.cs OK

```
1 const float MOVE_SPEED = 5f; // 1秒に進む距離
2
3 void Update()
4 {
5     float inputX = Input.GetAxis("Horizontal");
6     float speed = MOVE_SPEED * Time.deltaTime;
7     transform.Translate(inputX * speed, 0f, 0f);
8 }
```

何が起きるか

1フレームに 0.1 進むので、60fps なら1秒に 6、144fps なら 14.4。PC によって速さが変わる。

直し方

Time.deltaTime（前のフレームからの秒数）を掛けると「1秒に◯進む」という意味になる。

NG 08 ｜ 動きがおかしくなる

Rigidbody 付きを transform で動かす

C# PlayerMovement.cs NG

```
// Rigidbody2D が付いたプレイヤー
void Update()
{
    float inputX = Input.GetAxis("Horizontal");
    float speed = MOVE_SPEED * Time.deltaTime;
    transform.Translate(inputX * speed, 0f, 0f);
}
```

C# PlayerMovement.cs OK

```
[SerializeField] Rigidbody2D body;

void Update()
{
    float inputX = Input.GetAxis("Horizontal");
    Vector2 velocity = body.linearVelocity;
    velocity.x = inputX * MOVE_SPEED;
    body.linearVelocity = velocity;
}
```

何が起きるか

物理演算を無視してワープするように動く。壁にめり込む・すり抜ける・ガタガタ震える。

直し方

Rigidbody の速度や力で動かす。速度は「1秒あたり」なので deltaTime は掛けない。

NG 09 ｜ 動きがおかしくなる

力をかけ続ける処理を Update に書く

C# PlayerJetpack.cs NG

```
1 [SerializeField] Rigidbody2D body;
2
3 void Update()
4 {
5     if (Input.GetKey(KeyCode.Space))
6     {
7         body.AddForce(Vector2.up * THRUST_POWER);
8     }
9 }
```

C# PlayerJetpack.cs OK

```
1 [SerializeField] Rigidbody2D body;
2 bool isThrusting;
3
4 void Update()
5 {
6     isThrusting = Input.GetKey(KeyCode.Space);
7 }
8 void FixedUpdate()
9 {
10     if (isThrusting)
11         body.AddForce(Vector2.up * THRUST_POWER);
12 }
```

何が起きるか

Update の回数は PC で変わる。毎回力を足すと、PC によって上がる速さが変わってしまう。

直し方

入力は Update で受け取って変数に入れる。力をかけ続ける処理は FixedUpdate に書く。

NG 10 ｜ 動きがおかしくなる

GetKeyDown を FixedUpdate で読む

C# PlayerJump.cs NG

```
1 void FixedUpdate()
2 {
3     if (Input.GetKeyDown(KeyCode.Space))
4     {
5         Jump();
6     }
7 }
```

C# PlayerJump.cs OK

```
1 void Update()
2 {
3     // 押した瞬間は Update で受け取る
4     if (Input.GetKeyDown(KeyCode.Space))
5     {
6         Jump(); // 1回だけの Impulse
7     }
8 }
```

何が起きるか

GetKeyDown は押したフレームだけ true。FixedUpdate はそのフレームに呼ばれないことがあり、入力を見逃す。

直し方

入力は必ず Update で読む。1回きりのジャンプ（Impulse）なら Update から呼んでよい。

NG 11 ｜ 動きがおかしくなる

while で「待つ」

C# StageStarter.cs NG

```
1 void Start()
2 {
3     float endTime = Time.time + 3f;
4     while (Time.time < endTime)
5     {
6         // 3秒待つつもり
7     }
8     StartStage();
9 }
```

C# StageStarter.cs OK

```
1 const float START_DELAY = 3f;
2
3 void Start()
4 {
5     StartCoroutine(StartStageAfterDelay());
6 }
7
8 IEnumerator StartStageAfterDelay()
9 {
10     yield return new WaitForSeconds(START_DELAY);
11     StartStage();
12 }
```

何が起きるか

while が終わるまで次のフレームに進まず、Time.time も増えない。永遠に終わらず Unity ごと固まる。

直し方

待ちたいときはコルーチンを使う。yield return で「続きは次のフレーム以降」にできる。

PART 3 | NG 12 – 16

文字列で指定する書き方

3

名前を文字列で書くと、打ち間違えや名前の変更に気づけない

NG 12 ｜ 重くなる・壊れやすい

タグを == で比べる

C# PlayerHealth.cs NG

```
1 void OnTriggerEnter2D(Collider2D other)
2 {
3     if (other.tag == "Enemy")
4     {
5         TakeDamage(ENEMY_DAMAGE);
6     }
7 }
```

何が起きるか

.tag は読むたびに新しい文字列を作る。"Enemey" と打ち間違えてもエラーにならず、ずっと false。

C# PlayerHealth.cs OK

```
1 void OnTriggerEnter2D(Collider2D other)
2 {
3     if (other.CompareTag("Enemy"))
4     {
5         TakeDamage(ENEMY_DAMAGE);
6     }
7 }
```

直し方

CompareTag を使う。文字列を作らず、存在しないタグを渡すとエラーで教えてくれる。

NG 13 ｜ 壊れやすい

GameObject の名前で判定する

C# PlayerHealth.cs NG

```
1 void OnCollisionEnter2D(Collision2D collision)
2 {
3     if (collision.gameObject.name == "Enemy")
4     {
5         TakeDamage(ENEMY_DAMAGE);
6     }
7 }
```

C# PlayerHealth.cs OK

```
1 void OnCollisionEnter2D(Collision2D collision)
2 {
3     if (collision.gameObject.CompareTag("Enemy"))
4     {
5         TakeDamage(ENEMY_DAMAGE);
6     }
7 }
```

何が起きるか

Prefab から生成すると名前は「Enemy(Clone)」になり一致しない。名前を変えただけでも壊れる。

直し方

「それが何者か」はタグで判定する。名前は人が見て分かるためのもので、判定には使わない。

NG 14 | 壊れやすい

メソッド名を文字列で渡す

C# PlayerRespawn.cs NG

```
1 void OnDead()
2 {
3     Invoke("Respawn", 3f);
4 }
5
6 void Respawn()
7 {
8     transform.position = respawnPoint.position;
9 }
```

C# PlayerRespawn.cs OK

```
1 const float RESPAWN_DELAY = 3f;
2
3 void OnDead()
4 {
5     Invoke(nameof(Respawn), RESPAWN_DELAY);
6 }
7
8 void Respawn()
9 {
10     transform.position = respawnPoint.position;
11 }
```

何が起きるか

メソッド名を変えても文字列は変わらず、エラーも出ないまま呼ばれなくなる。SendMessage も同じ。

直し方

nameof(Respawn) と書けば名前の変更についてきて、間違えるとコンパイルエラーになる。

NG 15 ｜ 壊れやすい

子オブジェクトを名前や順番で探す

C# PlayerShooter.cs NG

```
Transform muzzle;
Transform hpBar;

void Start()
{
    muzzle = transform.GetChild(0);
    hpBar = transform.Find("Canvas/HpBar");
}
```

C# PlayerShooter.cs OK

```
// Inspector でドラッグして設定する
[SerializeField] Transform muzzle;
[SerializeField] Transform hpBar;
```

何が起きるか

Hierarchy の並び順や名前を変えるだけで、別のものを取ったり null になったりする。

直し方

子オブジェクトは Inspector で直接つなぐ。Prefab の中の参照は Prefab と一緒に保存される。

NG 16 ｜ 重くなる・壊れやすい

Animator のパラメータを毎回文字列で渡す

C# PlayerAnimation.cs NG

```
1 [SerializeField] Animator animator;
2
3 void Update()
4 {
5     animator.SetBool("isRunning", isRunning);
6     animator.SetBool("isJumping", isJumping);
7 }
```

C# PlayerAnimation.cs OK

```
1 static readonly int IS_RUNNING =
2     Animator.StringToHash("isRunning");
3 static readonly int IS_JUMPING =
4     Animator.StringToHash("isJumping");
5
6 void Update()
7 {
8     animator.SetBool(IS_RUNNING, isRunning);
9     animator.SetBool(IS_JUMPING, isJumping);
10 }
```

何が起きるか

文字列は毎回番号に変換されている。打ち間違えても警告が出るだけで、止まらずに動かない。

直し方

最初に1回だけ番号（ハッシュ）に変換して使う。パラメータ名の文字列も1か所にまとまる。

PART 4 | NG 17 – 20

書き方の習慣

4

今は動いていても、作品が大きくなるほど直しにくくなる

NG 17 ｜ 壊れやすい

Inspector に出したいだけで public にする

C# PlayerHealth.cs NG

```
1 public int hp = 3;
2
3 // 別のスクリプトから、どこでも書きかえられる
4 playerHealth.hp -= 1;
5 playerHealth.hp = 999;
```

C# PlayerHealth.cs OK

```
1 [SerializeField] int maxHp = 3;
2 int currentHp;
3 bool isInvincible;
4
5 public void TakeDamage(int damage)
6 {
7     if (isInvincible) return;
8     currentHp -= damage;
9 }
```

何が起きるか

どのスクリプトからでも書きかえられる。HP がおかしくなったとき、どこで変えたのか探せない。

直し方

Inspector に出すだけなら [SerializeField] private。外から変えるときはメソッドを通す。

NG 18 ｜ 壊れやすい

1つのスクリプトに全部書く

NG

PlayerController.cs

移動 ・ ジャンプ ・ 攻撃
HP とダメージ ・ 無敵時間
アニメーション ・ 効果音
スコア ・ UI の表示 ・ ゲームオーバー

→ 何百行にもなる

→

PlayerMovement
移動・ジャンプ

PlayerAttack
攻撃

PlayerHealth
HP・ダメージ・無敵時間

PlayerAnimation
アニメーション・効果音

ScoreCounter
スコアの計算

ScoreDisplay
スコアの表示

何が起きるか

1か所直すと別の処理が壊れる。読む場所が多すぎてバグが探せない。チームで同時に編集できない。

直し方

「何をするか」が名前で言えるサイズに分ける。名前に迷ったら、分けどきのサイン。

NG 19 ｜ 壊れやすい

Start の順番に頼って、ほかの値を使う

C# 2つのスクリプト NG

```
// EnemySpawner.cs
void Start()
{
    spawnPoints = FindSpawnPoints();
}

// EnemyCounter.cs
void Start()
{
    maxEnemyCount = spawner.GetSpawnPointCount();
}
```

C# 2つのスクリプト OK

```
// EnemySpawner.cs（自分の準備）
void Awake()
{
    spawnPoints = FindSpawnPoints();
}

// EnemyCounter.cs（ほかを使う）
void Start()
{
    maxEnemyCount = spawner.GetSpawnPointCount();
}
```

何が起きるか

別々のオブジェクトの Start がどの順で呼ばれるかは決まっていない。たまたま動いても後で null になる。

直し方

自分の準備は Awake、ほかを使うのは Start。全員の Awake が終わってから Start が呼ばれる。

NG 20 ｜ 動きがおかしくなる

小数（座標）を == で比べる

C# PlayerGroundCheck.cs NG

```
1 void Update()
2 {
3     if (transform.position.y == 0f)
4     {
5         isGrounded = true;
6     }
7 }
```

C# PlayerGroundCheck.cs OK

```
1 const float GROUND_Y = 0f;
2 const float TOLERANCE = 0.01f;
3
4 void Update()
5 {
6     float offsetY = transform.position.y - GROUND_Y;
7     isGrounded = Mathf.Abs(offsetY) < TOLERANCE;
8 }
```

何が起きるか

小数は計算のたびに小さな誤差が出る。物理で落ちてきた y は、ぴったり 0 にはまずならない。

直し方

「差が十分に小さいか」で判定する。接地判定なら当たり判定や Raycast を使うほうが確実。